AI로 쇼핑몰 만들기

잘 묻고, 반드시 검증한다

3인 · 20분 · 2026-08

1. AI를 어떻게 썼나 — 프롬프트 방법론 · 문서 체계
2. AI로 무엇을 만들었나 — 주문 · 결제 · 상품 · 회원
3. AI 결과를 어떻게 검증했나 — 보안 · 품질

PART 1

AI를 어떻게 썼나

프롬프트 방법론 · 문서 체계

발표자 1 · 8분

무엇을 만들었나

26
고객 화면

33
관리자 화면

110
API

960
자동 테스트

> 한국형 온라인 쇼핑몰 — 고객몰 · 관리자 · 백엔드

오늘 할 얘기는
“쇼핑몰을 만들었다”가 아닙니다

AI를 어떻게 써서 만들었나

AI 쓰는 방식이 네 번 바뀌었습니다

6월 중순 시작 → 8월 초 · 약 7주

1기

그냥 시켰다

고치는 시간이
짜는 시간보다 길었다

→

2기

공식을 정했다

코드는 좋아졌는데
헛구현은 그대로

→

3기

함정을 미리 심었다

같은 실수를
다시 밟지 않게 됨

→

4기

문서를 나눴다

문서가 쌓여도
정확도가 안 떨어짐

> 처음부터 이렇게 한 게 아닙니다. **각 단계는 앞 단계가 깨져서 생겼습니다.**

1기 — 그냥 시켰습니다

우리가 한 말

상품 페이지 만들어줘

돌아온 것

· 매번 다른 구조
· 프로젝트 규칙 무시
· 로딩 · 빈 화면 · 에러 처리 없음

받아서 고치는 시간이 직접 짜는 시간보다 길었습니다. 여기서 방식을 바꿨습니다

2기 — 프롬프트에 공식을 만들었습니다

좋은 프롬프트 = 5요소

- ① **맥락** — 어떤 프로젝트·스택·디렉터리인지
- ② **할 일 (하나)** — 화면 1개 또는 함수 1개. 여러 개 절대 X
- ③ **따를 패턴** — "옆에 있는 OO처럼" 기존 코드를 지목
- ④ **출력 형태** — 타입 먼저 / diff / 파일:라인
- ⑤ **검증 요구** — "끝나면 타입체크하고 화면 확인까지"

같은 요청, 다른 결과

❌ 나쁜 예

상품 페이지 만들어줘

✅ 좋은 예

`products/[handle]` 페이지에 리뷰 탭 1개 추가해줘.
데이터는 `lib/data/reviews.ts` 경유
(없으면 mock+타입 먼저 제안).
옆 상세 섹션 패턴 그대로 따르고,
로딩/빈/에러 상태 처리.
먼저 접근을 제안하고 내 확인을 받은 뒤 구현해.
끝나면 `npx tsc --noEmit` 로 타입체크까지.

그런데 공식만으로는 안 됐습니다

프롬프트는 좋아졌는데, 크게 시키면 여전히 무너졌다

"주문 워크플로우 전체 만들어줘"

→ 코드는 나왔다
→ 그런데 어디가 틀렸는지 알 수가 없었다
→ 되돌리고 처음부터

덩어리가 크면 **검증할 수가 없습니다.** 그래서 규칙을 세 개 더 붙였습니다

규칙 세 개를 붙였습니다

- **① 접근을 먼저 제안하게** — 바로 코드부터 뽑지 못하게
  30분짜리 삽질을 30초 대화로 막는다
- **② 한 프롬프트 = 한 기능** — 재고 / 할인 / 적립금을 따로
  단계마다 테스트하고 넘어갈 수 있다
- **③ 계약(타입)을 먼저** — 타입 → mock → 화면 → 실 API
  타입만 맞춰두면 서로 안 기다려도 된다

> ①은 헛구현을, ②는 "검증 불가"를, ③은 서로 기다리는 시간을 없앴습니다

3기 — 겪은 실수를 프롬프트에 심었습니다

고친 뒤 그냥 넘어가지 않고, 다음 프롬프트에 경고로 박았다

```
mock 에 Math.random() · new Date() 쓰지 마
   → 하이드레이션 에러가 난다

items.total 같은 계산 필드를 단독 선택하지 마
   → null 이나 절반 값으로 온다

dev 서버 켠 채로 build 하지 마
   → 실행 중인 화면이 500 이 된다
```

> AI는 같은 함정을 **매번 다시 밟습니다.** 한 번 겪을 때마다 문서에 적고 프롬프트에 붙였더니 안 밟더군요

4기 — 문서를 AI에게 줬더니 결과가 나빠졌습니다

문서가 70개까지 쌓인 시점

사람용 문서

"왜 그렇게 했나"
비유 · 배경 · 맥락
읽기 좋게

AI용 문서

"정확히 무엇인가"
타입 · 경로 · 제약
토큰 효율

> 비유와 배경이 토큰만 잡아먹고, 정작 필요한 타입 정의는 묻힌다

★ 문서를 독자별 4층으로 나눴습니다

폴더	독자	형식
01_팀원용	사람 (합류자)	설명 · 비유 · "왜 그렇게 했나"
02_AI용	AI	사실 나열 · 정확한 타입과 경로
03_운영용	사장님 · 운영자	개발 지식 0 가정
04_점검기록	검증하는 사람	결과 + 재현 가능한 실행 도구

> 02_AI용 문서는 "이 문서를 읽는 AI는 …"으로 시작한다. 사람이 읽으라고 쓴 게 아니다

★ 네 번 바꾸고 나서

바꾼 것	달라진 것
1기 → 2기 프롬프트 5요소	프로젝트 규칙을 지킨 코드가 나오기 시작
2기 → 3기 접근 먼저 · 한 기능씩 · 계약 먼저	되돌리는 일이 줄고, 병렬 작업이 가능해짐
3기 → 4기 겪은 함정을 프롬프트에 주입	커스텀 모듈 **27개가 같은 구조**로 유지됨
4기 문서를 독자별 4층으로	문서 **70개**를 쌓아도 답변 정확도가 안 떨어짐

> 앞 두 줄은 **체감**이고, 뒤 두 줄은 **결과물로 확인**됩니다 — 모듈 27개 · 문서 4층

정리

- **처음부터 잘 쓴 게 아닙니다** — 네 번 깨지고 네 번 바꿨습니다
- **깨질 때마다 규칙으로 남겼습니다** — 사람이 기억하는 게 아니라 프롬프트·문서에
- **다음 두 분이 이 방식으로 만든 것과 검증한 것을 보여드립니다**

PART 2

AI로 무엇을 만들었나

주문 · 결제 · 상품 · 회원

발표자 2 · 7분

주문은 “구매 버튼 누르면 끝”이 아닙니다

- 재고 확인 · 차감
- 쿠폰 할인 · 적립금 사용
- 배송비 계산
- 주문 생성 · 결제 대기
- **중간에 하나라도 실패하면 전부 롤백**

> 이런 건 바로 코드부터 뽑으면 안 된다

그래서 “접근 먼저” 로 물었습니다

```
[맥락] Medusa v2 백엔드. 한국 쇼핑몰 주문 흐름 설계.

[할 일] 장바구니 → 결제 대기까지의 워크플로우
  1) 재고 확인·차감 (동시 주문에도 안전하게)
  2) 쿠폰 할인    3) 적립금 사용
  4) 배송비       5) 주문 생성    6) 결제 대기
  * 중간 실패 시 앞 단계 전부 롤백

[진행] 바로 코드 짜지 말고, 단계별 흐름과
  입력·출력·실패 시 보상을 표로 제안해.
  내가 확인하면 구현.
```

돌아온 건 코드가 아니라 흐름표

```
1 장바구니 검증 → 2 재고 확인(동시성) → 3 할인 →
4 적립금 차감 → 5 배송비 → 6 주문 생성 →
7 재고 예약 → 8 결제 정보

각 단계 실패 시 앞 작업을 자동 보상
```

> "여기 순서 이상한데?"를 우리가 짚을 수 있었다. 코드가 나온 뒤였으면 다 갈아엎어야 했다

걱정되면 근거를 요구했습니다

동시 주문

재고 1개인데 두 명이 동시에 주문하면?

추측하지 말고 Medusa 재고 처리 코드를
확인해서 파일:라인으로 근거를 대줘.

할인 순서

쿠폰 10% + 적립금 5000원을 같이 쓰면
어느 걸 먼저 적용해야 하지?

순서에 따라 최종 금액이 달라지는지
계산해서 보여줘.

> "안전합니다"만 받았으면 넘어갔을 것 — 근거를 요구하니 실제 코드를 확인하고 답했다

한 프롬프트 = 한 기능

"주문 워크플로우 전체 만들어줘"는 실패했다

그래서 쪼갰습니다

- **1단계** 재고 확인만
- **2단계** 할인 계산만
- **3단계** 적립금 차감만
- **4단계** 합치기

> 각 단계마다 테스트하고 넘어갈 수 있었다.
> 통째로 던지면 그럴듯하지만 검증 불가능한 코드가 나온다

1단계 프롬프트

```
[할 일] 재고 확인 함수 하나만. 다른 단계는 아직 X.
[패턴] 옆에 있는 validateCart 함수 구조 그대로.
[출력] 타입 먼저 제안하고, 확인받은 뒤 구현.
[검증] 동시 요청 2건 테스트 코드도 같이.
```

상품·회원 — “옆에 있는 저것처럼”

❌ **알아서 잘**

상품 목록에 가격 필터 추가해줘

✅ **기존 코드를 지목**

상품 목록에 가격 필터 추가해줘.
옆에 있는 카테고리 필터(refinement-list/)
구조 그대로 따라가고,
URL 쿼리 파라미터 방식도 동일하게.

> 커스텀 모듈 27개가 전부 같은 구조 — 하나만 이해하면 나머지가 읽힌다

고치는 과정에서 다른 걸 깨뜨렸습니다

- **보안 패치가 린터를 죽였다**
  취약점 대응으로 넣은 버전 고정이 ESLint 가 쓰는 라이브러리까지 올려서, 린트가 아예 안 돌았다

- **첫 설계가 프레임워크에 막혔다**
  재고 강제 차감을 `?force=true` 쿼리로 만들었더니 코어 검증에 걸려 400. 헤더 방식으로 바꿔 해결

- **최적화를 켰더니 페이지가 죽었다**
  이미지 최적화 ON → 등록 안 된 이미지 호스트 때문에 상품 상세가 500

> 셋 다 **실제로 돌려보고** 알았습니다. 그래서 "끝나면 검증까지"가 프롬프트에 들어갔습니다

정리

- **복잡한 건 설계부터** — 코드 전에 흐름표
- **걱정되면 근거를 요구** — "추측하지 마"
- **한 프롬프트 = 한 기능** — 쪼개야 검증할 수 있다
- **고치면 옆이 깨진다** — 그래서 매번 돌려봐야 한다

PART 3

AI 결과를 어떻게 검증했나

보안 · 품질

발표자 3 · 7분

AI는 그럴듯하게 틀립니다

코드는 돌아가고, 화면도 뜨고, 테스트도 통과한다. 그런데 틀렸다

검증을 세 층으로 깔았습니다

- ① **타입체크** — 컴파일 단계
- ② **자동 테스트 960개** — 계산 로직 · 화면 흐름
- ③ **보안 프로브 52건** — 실제 서버에 공격 요청을 던진다

> 이 중 ③번이 진짜를 잡았다

① 적립금을 세 배로 쓸 수 있었습니다

장바구니 3개에 10,000원 적립금을 동시에 사용

```
카트1: 200 OK  →  10,000원 할인
카트2: 200 OK  →  10,000원 할인
카트3: 200 OK  →  10,000원 할인

보유 10,000원인데  30,000원어치 할인
잔액: -20,000원
```

원인 — 락을 잘못된 단위로 잡았다

- 락 키가 **"장바구니별"**이었다
- 그런데 적립금 잔액은 **"고객별" 자산**
- 장바구니를 여러 개 만들면 서로 다른 락 → 의미 없음

> 주석에는 "동시 차감 방지"라고 적혀 있었다. 코드만 읽으면 안전해 보인다

② 환불이 세 번 나갔습니다

같은 요청번호로 1,000원 환불을 동시에 3번

```
3건 모두 성공
DB 환불 기록: 3건 (각 1,000원)

→ 1,000원 요청했는데  3,000원이 실제로 나감
```

> 순차로 보내면 정상적으로 막힌다. 동시에 보낼 때만 뚫린다 — 그래서 발견이 안 됐다

셋 다 원인이 같았습니다

```
검사  →  (락 없음)  →  집행
              ↑
        여기로 다른 요청이 끼어든다
```

> 고친 뒤 같은 공격을 다시 던져 확인 — 동시 5건 → 성공 1건, 잔액 정확히 0

저희 점검도 틀렸습니다

"완료"라고 적어둔 것이 틀린 경우

- **주석에는 "이중차감 방지"라고 적혀 있었습니다**
  일주일 전 점검에서 통과 처리한 항목이었다. 락을 걸긴 걸었는데 단위가 틀렸다
- **환불엔 고쳤는데 취소엔 반영을 안 했습니다**
  같은 원인, 다른 파일. 한 곳만 고치고 끝냈다
- **"화면이 안 바뀌면 고장"이라는 판정 기준이 오탐을 냈습니다**
  멀쩡한 기능 4개를 고장으로 표시했다

> 그래서 판정 기준을 **항목별로** 다시 잡았습니다 — URL · 쿠키 · 네트워크 요청 · 특정 요소

③ 소셜 로그인 — 모두가 같은 계정

브라우저 두 개로 각각 로그인

```
방문자 A  →  kakao-demo@shop.local
방문자 B  →  kakao-demo@shop.local     ← 같은 계정!
```

> 이대로 열었으면 고객 B가 고객 A의 주문내역·배송지·연락처를 그대로 봤을 것

검증을 자산으로 만들었습니다

찾은 걸 문서로만 남기지 않았다

- `presale-security-probe.mjs` — 27건 · 권한 · 인젝션 · 헤더
- `audit2-probe.mjs` — 19건 · 토큰위조 · 권한상승 · 입력경계
- `audit3-concurrency-probe.mjs` — 6건 · **동시 이중집행**

> 명령 한 줄이면 지금도 확인된다. 다시 생기면 바로 잡힌다

★ 숫자로 본 결과

항목	고치기 전	고친 뒤
적립금 동시 사용	10,000P로 30,000원 할인	동시 10건 → 성공 5건, 잔액 0
환불 동시 요청	3건 집행 (3,000원)	1건 집행 · 4건 거부
백엔드 린트	설정 자체가 없었음	218건 검출 → 0 error
취약점 (high)	스토어프론트 9 · 어드민 7	0 · 0
히어로 이미지	928KB PNG	11KB AVIF
관리자 죽은 버튼	17개	1개 연결 · 16개 비활성+사유

> 백엔드 high 4건은 **일부러 안 고쳤습니다** — 도달 불가를 실측으로 확인하고 이유를 문서에 남겼습니다.
> "전부 0" 보다 **왜 남겼는지 설명되는 것**이 중요합니다

검증하지 않았으면
돈이 새는 걸 몰랐을 겁니다

코드만 읽으면 안전해 보였고, 주석에도 "방지함"이라고 적혀 있었다

잘 묻고,
반드시 검증한다

저희가 배운 건 이거였습니다